Type double and long double

- The larger floating point types, double and long double, are similar to float except that they require more memory space and provide a wider range of values and more precision.
- Type double requires 8 bytes of storage and handles numbers in the range from 1.7x10^–308 to 1.7x10^308. Type long double is compiler-dependent but is often the same as double.

Floating-Point Constants

- The number 3.14159F is an example of a *floating-point constant*.
- The decimal point signals that it is a floating-point constant, and not an integer, and the F specifies that it's type float, rather than double or long double.
- The number is written in normal decimal notation. You don't need a suffix letter with constants of type double; it's the default. With type long double, use the letter L.

Contdd . . . .

- You can also write floating-point constants using *exponential notation*.
- Exponential notation is a way of writing large numbers without having to write out a lot of zeros.
- For example,1,000,000,000 can be written as 1.0E9 in exponential notation.

The const Qualifier

- The keyword final (for constant) precedes the data type of a variable.
- It specifies that the value of a variable will not change throughout the program.
- Any attempt to alter the value of a variable defined with this qualifier will elicit an error message from the compiler.

Type bool

- Variables of type bool can have only two possible values: true and false
- Type bool is most commonly used to hold the results of comparisons.
- For example:

Is var1 less than var2?

If so, a bool value is given the value true;

If not, it's given the value false.

Variable Type Summary

Keyword	Numerical Range Low	Numerical Range High	Digits of Precision	Bytes of Memory
`bool`	false	true	n/a	1
`char`	–128	127	n/a	1
`short`	–32,768	32,767	n/a	2
`int`	–2,147,483,648	2,147,483,647	n/a	4
`long`	–2,147,483,648	2,147,483,647	n/a	4
`float`	3.4×10^{-38}	3.4×10^{38}	7	4
`double`	1.7×10^{-308}	1.7×10^{308}	15	8

Unsigned Data Types

Keyword	Numerical Range Low	Numerical Range High	Bytes of Memory
unsigned char	0	255	1
unsigned short	0	65,535	2
unsigned int	0	4,294,967,295	4
unsigned long	0	4,294,967,295	4

Type Conversion

```c
#include<stdio.h>

void main(void){
    int count = 7;
    float avgWeight = 155.5F;

    double totalWeight = count * avgWeight;
    printf("total weight = %lf", totalWeight );
}
```

Automatic Conversions

Data Type	*Order*
`long double`	Highest
`double`	
`float`	
`long`	
`int`	
`short`	
`char`	Lowest

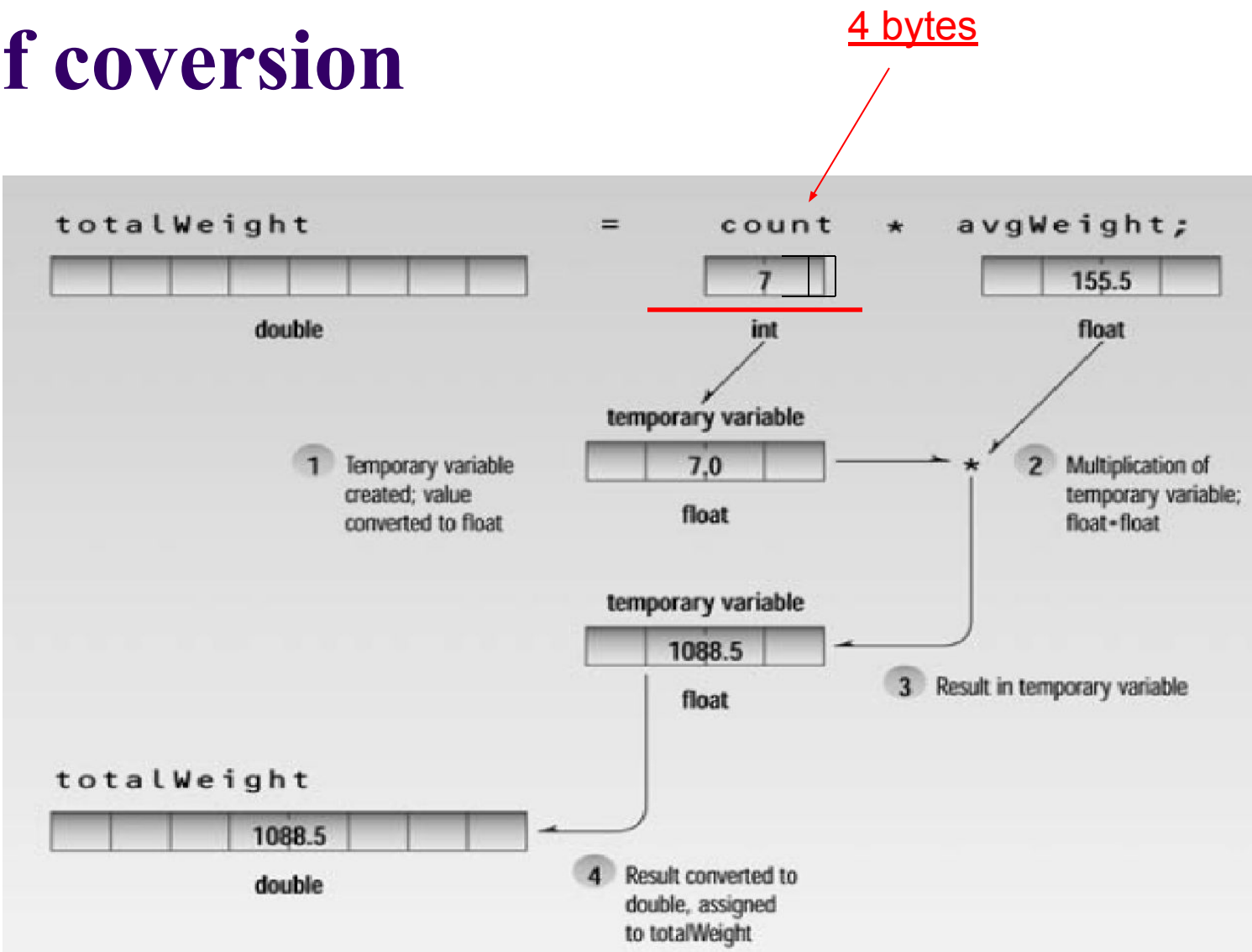
Automatic Conversions

- When two operands of different types are encountered in the same expression, the lower-type variable is converted to the type of the higher-type variable.

Automatic Conversions

- In our example, the int value of count is converted to type float and stored in a temporary variable before being multiplied by the float variable avgWeight.
- The result (still of type float) is then converted to double so that it can be assigned to the double variable totalWeight.

Process of coversion



These conversions take place invisibly

Casts

- In C Cast applies to data conversions specified by the programmer, as opposed to the automatic data conversions.
- Casts are also called *type casts*.

What are casts for?

Casts

- Sometimes a programmer needs to convert a value from one type to another in a situation where the compiler will not do it automatically or without complaining.

Example

```c
#include<stdio.h>

void main(void){
    int count = 7;
    float avgWeight = 155.5F;

    int totalWeight = (int)count * avgWeight;
    printf("total weight = %d", totalWeight );
}
```

QUESTIONS????